Docker Full Notes — Beginner to Advanced

বাংলা + English | Python Django Backend Developer

---

1. What is Docker?

Docker is a platform used to build, package, distribute, and run applications inside containers.

Docker application-এর code, dependencies এবং configuration একটি portable image-এ package করতে সাহায্য করে।

Why Docker?

- Portability
- Environment consistency
- Application isolation
- Faster deployment
- Easy scaling
- CI/CD integration
- Simplified development setup

Example

A Django project requires:

- Python 3.13
- Django
- PostgreSQL
- Python dependencies
- Environment variables

Docker ব্যবহার করলে application-এর environment সহজে পুনরায় তৈরি করা যায়।

Important: Containers সাধারণত host operating system-এর kernel share করে। Container একটি সম্পূর্ণ virtual machine নয়।

---

2. Docker Architecture

Docker-এর প্রধান components:

1. Docker Client
2. Docker Daemon
3. Docker Images

4. Docker Containers
5. Docker Registry
6. Docker Networks
7. Docker Volumes
8. Docker Compose

Docker workflow

Dockerfile → Docker Image → Docker Container → Running Application

Docker Client

Docker CLI দিয়ে commands চালানো হয়।

Example:

```
docker ps
docker images
docker run nginx
```

Docker Daemon

Docker Engine-এর background service, যা images, containers, networks এবং volumes manage করে।

Docker Registry

Docker images সংরক্ষণ ও বিতরণ করার service।

Examples:

- Docker Hub
- GitHub Container Registry
- Amazon Elastic Container Registry

---

3. Docker Installation

Official documentation:

https://docs.docker.com/get-started/

Windows-এর জন্য সাধারণত Docker Desktop install করতে হয়। Linux-এর জন্য Docker Engine ব্যবহার করা যায়।

Verify installation

```
docker --version
docker compose version
docker info
docker run hello-world
```

---

4. Docker Image vs Container

Docker Image

Image হলো application package বা template, যা থেকে container তৈরি হয়।

Example:

```
docker pull python:3.13-slim
docker images
```

Docker Container

Container হলো image থেকে তৈরি runtime instance।

Example:

```
docker run nginx
docker ps
```

Differences

Image	Container
Template/ package	Runtime instance
Image layers সাধারণত read-only	Writable layer থাকতে পারে
Image থেকে container তৈরি হয়	Image ব্যবহার করে চলে
`docker images`	`docker ps`

---

5. Essential Docker Commands

Image Commands

```
docker images
docker image ls
docker pull nginx
docker pull python:3.13-slim
docker image inspect nginx
docker image rm nginx
docker image prune
```

Container Commands

```
docker ps
docker ps -a
docker run nginx
docker start CONTAINER_NAME
docker stop CONTAINER_NAME
docker restart CONTAINER_NAME
docker rm CONTAINER_NAME
docker logs CONTAINER_NAME
docker logs -f CONTAINER_NAME
docker inspect CONTAINER_NAME
docker stats
```

Interactive Commands

```
docker exec -it CONTAINER_NAME sh
docker exec -it CONTAINER_NAME bash
```

`bash` সব image-এ available নয়। Bash না থাকলে `sh` ব্যবহার করো।

Run Commands

```
docker run --rm alpine echo "Hello Docker"

docker run -d --name mynginx nginx

docker run -d -p 8080:80 --name web nginx

docker run -it --rm python:3.13-slim python
```

Important flags

- `-d` : Background mode

- `-it` : Interactive terminal
- `-p` : Port mapping
- `-v` : Volume mount
- `--name` : Container name
- `--rm` : Exit হলে container automatically remove
- `--network` : Network select
- `-e` : Environment variable
- `--restart` : Restart policy

---

6. Dockerfile

Dockerfile হলো image build করার instruction file।

Example Django Dockerfile

```
FROM python:3.13-slim

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1
ENV PIP_NO_CACHE_DIR=1

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "manage.py", "runserver", "0.0.0.0:8000"]
```

এই example development-এর জন্য। Production-এ Gunicorn বা উপযুক্ত production application server ব্যবহার করবে।

Dockerfile instructions

Instruction	Purpose

`FROM`	Base image
`WORKDIR`	Working directory
`COPY`	Files copy
`ADD`	Copy-এর অতিরিক্ত কিছু সুবিধা
`RUN`	Build-time command
`ENV`	Environment variable
`ARG`	Build-time variable
`EXPOSE`	Intended container port document করে
`CMD`	Default runtime command
`ENTRYPOINT`	Main executable
`USER`	Runtime user

`VOLUME`	Mount point declare করে
`HEALTHCHECK`	Health check configuration
`LABEL`	Image metadata

RUN vs CMD vs ENTRYPOINT

RUN

Image build করার সময় চলে।

```
RUN pip install django
```

CMD

Container start হলে default command বা arguments দেয়।

```
CMD ["python", "manage.py", "check"]
```

ENTRYPOINT

Main executable নির্ধারণ করে।

```
ENTRYPOINT ["python", "manage.py"]
CMD ["check"]
```

এখন:

```
docker run myapp migrate
```

কার্যত `python manage.py migrate` চালাবে।

---

7. Build and Run Docker Image

Project root-এ Dockerfile থাকলে:

```
docker build -t django-app:1.0 .

docker images

docker run -d \
  --name django-container \
  -p 8000:8000 \
  django-app:1.0

docker ps

docker logs -f django-container
```

Browser:

```
http://localhost:8000
```

Container stop ও remove:

```
docker stop django-container
docker rm django-container
```

Image remove:

```
docker image rm django-app:1.0
```

---

8. Docker Image Layer and Build Cache

Docker images multiple layers দিয়ে তৈরি হয়।

Example:

```
FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .
```

প্রথমে `requirements.txt` copy ও dependencies install করা হয়েছে।

এরপর application code copy করা হয়েছে।

এতে source code পরিবর্তিত হলে dependencies layer সাধারণত reuse করা যায়।

Best practices

- Stable instructions আগে রাখো।
- Frequently changing files পরে copy করো।
- `.dockerignore` ব্যবহার করো।
- Unnecessary dependencies বাদ দাও।
- Trusted এবং maintained base image ব্যবহার করো।

---

9. .dockerignore

`.dockerignore` file build context থেকে unnecessary files বাদ দেয়।

Example:

```
.git
.gitignore
.env
.venv
venv/
__pycache__/
*.py[cod]
.pytest_cache/
.mypy_cache/
.coverage
htmlcov/
node_modules/
media/
staticfiles/
```

Benefits:

- Smaller build context
- Faster build
- Sensitive files image-এ যাওয়ার ঝুঁকি কমানো
- Cleaner image build

Note: `.dockerignore` runtime bind mount-এর access control নয়। Production secrets আলাদাভাবে manage করতে হবে।

---

10. Docker Port Mapping

Command:

```
docker run -d -p 8080:80 nginx
```

এখানে:

- Host port: 8080
- Container port: 80

Browser:

```
http://localhost:8080
```

Syntax:

```
-p HOST_PORT:CONTAINER_PORT
```

`EXPOSE 8000` নিজে থেকে host port publish করে না।

---

11. Docker Volumes

Volume ব্যবহার করা হয় container-এর lifecycle থেকে আলাদা persistent storage রাখার জন্য।

Create volume

```
docker volume create mydata

docker volume ls

docker volume inspect mydata
```

Use volume

```
docker run -d \
  --name postgres_db \
  -e POSTGRES_DB=mydb \
  -e POSTGRES_USER=myuser \
  -e POSTGRES_PASSWORD=change-me \
  -v pgdata:/var/lib/postgresql/data \
  postgres:17
```

Production-এ demo password-এর পরিবর্তে secure secret ব্যবহার করবে।

Volume types

Named Volume

Docker-managed persistent storage।

Bind Mount

Host directory container-এ mount করা হয়।

```
docker run --rm \
  --mount type=bind,source=.,target=/app \
  python:3.13-slim
```

tmpfs

Linux-এ temporary memory-backed storage।

```
docker run --rm --tmpfs /tmp alpine
```

Important

Container remove করলেই named volume সাধারণত remove হয় না।

Volume backup করা প্রয়োজন। Persistent volume নিজে থেকে backup নয়।

---

12. Docker Networking

Docker networking container communication পরিচালনা করে।

Bridge network

```
docker network create mynetwork

docker run -d \
  --name web \
  --network mynetwork \
  nginx

docker run --rm \
  --network mynetwork \
  alpine ping -c 3 web
```

User-defined bridge network-এ container name ব্যবহার করে service discovery করা যায়।

Common network types

- Bridge
- Host
- None
- Overlay

Django database host

Django container থেকে PostgreSQL access করার সময় সাধারণত `localhost` নয়, service name ব্যবহার করতে হয়।

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "mydb",
        "USER": "myuser",
        "PASSWORD": "your-secret",
        "HOST": "db",
        "PORT": "5432",
    }
}
```

Docker Compose-এ PostgreSQL service-এর নাম `db` হলে hostname হবে `db`।

---

13. Docker Compose

Docker Compose YAML file দিয়ে একাধিক container, network এবং volume manage করে।

Typical Django stack:

- Django
- PostgreSQL
- Redis
- Celery worker

Project structure

```
django-project/
├── manage.py
├── requirements.txt
├── Dockerfile
├── compose.yaml
├── .dockerignore
├── .env
├── config/
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
└── app/
```

compose.yaml

```
services:
  web:
```

```
    build:
      context: .
    command: python manage.py runserver 0.0.0.0:8000
    ports:
      - "8000:8000"
    volumes:
      - .:/app
    env_file:
      - .env
    environment:
      DB_HOST: db
      DB_PORT: "5432"
    depends_on:
      db:
        condition: service_healthy
    restart: unless-stopped

  db:
    image: postgres:17
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test:
        - CMD-SHELL
        - pg_isready -U "$$POSTGRES_USER" -d "$$POSTGRES_DB"
      interval: 5s
      timeout: 5s
      retries: 10
    restart: unless-stopped

volumes:
  postgres_data:
```

.env

```
POSTGRES_DB=django_db
POSTGRES_USER=django_user
POSTGRES_PASSWORD=replace-with-a-strong-password
```

```
DB_NAME=django_db
DB_USER=django_user
DB_PASSWORD=replace-with-a-strong-password
```

`.env` Git-এ commit করবে না।

Django database settings

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": os.environ["DB_NAME"],
        "USER": os.environ["DB_USER"],
        "PASSWORD": os.environ["DB_PASSWORD"],
        "HOST": os.environ.get("DB_HOST", "db"),
        "PORT": os.environ.get("DB_PORT", "5432"),
    }
}
```

এখানে `import os` করতে হবে এবং PostgreSQL driver install থাকতে হবে।

Compose commands

```
docker compose config

docker compose up -d --build

docker compose ps

docker compose logs -f

docker compose logs -f web

docker compose exec web sh

docker compose exec web python manage.py migrate

docker compose exec web python manage.py createsuperuser

docker compose restart web

docker compose down
```

down vs down -v

```
docker compose down
```

Containers এবং Compose network সরায়, named volume সাধারণত রেখে দেয়।

```
docker compose down -v
```

Managed volumes-ও remove করতে পারে। Database data হারানোর ঝুঁকি আছে।

---

14. Docker Environment Variables

Environment variables দিয়ে configuration ও sensitive settings source code থেকে আলাদা রাখা যায়।

```
docker run -e DEBUG=False myapp
```

Compose-এ:

```
services:
  web:
    environment:
      DEBUG: "False"
      DB_HOST: db
```

অথবা:

```
services:
  web:
    env_file:
      - .env
```

Important distinction

- Compose `.env` : YAML variable interpolation-এ ব্যবহার হয়।
- `env_file` : Container-এর environment-এ variables সরবরাহ করে।
- `environment` : নির্দিষ্ট environment variables সরবরাহ করে।

Sensitive production secrets-এর জন্য secret manager বা deployment platform-এর secrets feature ব্যবহার করো।

---

15. Docker Registry and Docker Hub

Registry images store ও distribute করে।

Login

```
docker login
```

Build and tag

```
docker build -t myusername/django-api:1.0 .
```

Push

```
docker push myusername/django-api:1.0
```

Pull and run

```
docker pull myusername/django-api:1.0

docker run -d \
  -p 8000:8000 \
  myusername/django-api:1.0
```

Production-এ versioned tags অথবা immutable image digest ব্যবহার করা ভালো।

---

16. Multi-stage Docker Build

Multi-stage build দিয়ে build environment এবং runtime environment আলাদা করা যায়।

```
FROM python:3.13-slim AS builder

WORKDIR /build

COPY requirements.txt .

RUN pip install --no-cache-dir \
    --prefix=/install \
    -r requirements.txt


FROM python:3.13-slim AS runtime

ENV PYTHONDONTWRITEBYTECODE=1
ENV PYTHONUNBUFFERED=1

WORKDIR /app

COPY --from=builder /install /usr/local

COPY . .
```

```
RUN useradd --create-home appuser \
    && chown -R appuser:appuser /app

USER appuser

EXPOSE 8000

CMD ["gunicorn", "config.wsgi:application", "--bind", "0.0.0.0:8000"]
```

Requirements-এ Gunicorn থাকতে হবে।

`config.wsgi:application` নিজের Django project-এর WSGI module দিয়ে replace করবে।

Benefits

- Smaller runtime image
- Fewer unnecessary build tools
- Cleaner production environment
- Better security posture

Native dependencies থাকলে runtime stage-এও প্রয়োজনীয় libraries install করতে হবে।

---

17. Docker Health Check

Health check দিয়ে application বা service-এর health status monitor করা যায়।

Example:

```
HEALTHCHECK --interval=30s \
  --timeout=5s \
  --start-period=10s \
  --retries=3 \
  CMD python -c "import urllib.request;
urllib.request.urlopen('http://127.0.0.1:8000/health/')" || exit 1
```

Application-এ `/health/` endpoint থাকতে হবে।

Health check configuration container-এর health status নির্ধারণ করে। এটি নিজে থেকে সব ধরনের recovery বা traffic routing নিশ্চিত করে না।

---

18. Production Django Docker

Production-এর জন্য development server ব্যবহার না করে Gunicorn বা উপযুক্ত application server ব্যবহার করো।

Example command:

```
gunicorn config.wsgi:application --bind 0.0.0.0:8000
```

Production checklist:

- `DEBUG=False`
- Secure `SECRET_KEY`
- Correct `ALLOWED_HOSTS`
- HTTPS
- Non-root container user
- Pinned dependencies
- Trusted base images
- Persistent database storage
- Database backups
- Application logging
- Health checks
- Secrets management
- Static/media file strategy
- Controlled migrations
- Image vulnerability scanning

Production migration সাধারণত deployment pipeline-এর controlled one-off step হিসেবে চালানো ভালো।

---

19. Docker Troubleshooting

Port already allocated

কারণ: Host port অন্য process ব্যবহার করছে।

সমাধান:

```
docker ps
```

অথবা অন্য host port ব্যবহার করো:

```
docker run -p 8081:8000 myapp
```

Container exited

```
docker ps -a
docker logs CONTAINER_NAME
```

Django database connection error

Check:

- PostgreSQL container running কি না
- Database health status
- Correct credentials
- Correct database hostname
- PostgreSQL driver installed কি না

```
docker compose ps
docker compose logs db
docker compose logs web
```

ModuleNotFoundError

```
docker compose build --no-cache web
docker compose up -d web
```

Dependencies `requirements.txt`-এ আছে কি না যাচাই করো।

Inspect container

```
docker inspect CONTAINER_NAME
docker stats
docker system df
docker compose config
```

---

20. Docker Cleanup

```
docker container prune
docker image prune
docker network prune
docker builder prune
docker system df
```

Unused images ও build cache remove করার আগে প্রয়োজনীয় image আছে কি না যাচাই করো।

`docker system prune -a` অনেক unused image সরাতে পারে।

`docker volume prune` persistent data মুছে ফেলতে পারে। Database backup ছাড়া এটি ব্যবহার করবে না।

---

21. Docker Security Best Practices

1. Trusted base images ব্যবহার করো।

2. Dependencies নিয়মিত update করো।
3. Non-root user হিসেবে application চালাও।
4. Privileged mode এড়িয়ে চলো।
5. Secrets image-এর মধ্যে hardcode করবে না।
6. প্রয়োজন ছাড়া ports publish করবে না।
7. Host Docker socket container-এ mount করা এড়িয়ে চলো।
8. Resource limits ও logging configure করো।
9. Image vulnerability scanning চালাও।
10. Production image ছোট রাখো।
11. Database backup ও restore পরীক্ষা করো।
12. Container isolation-কে সম্পূর্ণ security boundary ধরে নেবে না।

---

22. Docker vs Virtual Machine

Docker Container	**Virtual Machine**
Host kernel share করে	Guest OS থাকে
সাধারণত দ্রুত startup	সাধারণত বেশি startup overhead
সাধারণত কম resource usage	সাধারণত বেশি resource usage
OS-level isolation	Hypervisor-based isolation
Application packaging	Complete OS

	environments

কিছু security-sensitive workload-এর জন্য VM-based isolation বেশি উপযুক্ত হতে পারে।

---

23. Docker vs Docker Compose vs Kubernetes

Tool	Purpose
Docker Engine	Containers ও images চালানো ও manage করা
Docker Compose	Multi-container applications configure করা
Kubernetes	Cluster-level orchestration
Docker Hub	Image registry
Container Registry	Images store ও distribute করা

শুরুতে Docker এবং Compose শেখো। তারপর প্রয়োজন অনুযায়ী Kubernetes শিখবে।

---

24. Docker Interview Questions

Q1. What is Docker?

Docker is a platform for building, packaging, and running applications in containers.

Q2. What is a Docker image?

An image is a packaged template used to create containers.

Q3. What is a Docker container?

A container is a runtime instance created from an image.

Q4. What is a Dockerfile?

A file containing instructions to build a Docker image.

Q5. RUN vs CMD?

RUN executes during image build. CMD specifies a default runtime command or arguments.

Q6. CMD vs ENTRYPOINT?

ENTRYPOINT defines the main executable. CMD provides default command or arguments depending on the configuration.

Q7. What is a Docker volume?

Persistent storage managed independently of a container's writable layer.

Q8. What is Docker Compose?

A tool for defining and running multi-container applications using YAML configuration.

Q9. Does EXPOSE publish a port?

No. Port publishing normally requires `-p` or Compose `ports`.

Q10. What does localhost mean inside a container?

It normally refers to the container itself.

Q11. Why use multi-stage builds?

To separate build tools from runtime dependencies and produce cleaner images.

Q12. Does depends_on guarantee application readiness?

Basic `depends_on` controls startup ordering. Health-based conditions can wait for a dependency to become healthy, but runtime retry logic may still be necessary.

Q13. Does removing a container delete its named volume?

Usually not. However, removing the volume itself can delete its data.

Q14. Why avoid Django runserver in production?

It is intended for development. Production should use an appropriate application server and deployment configuration.

Q15. How do you debug a container?

Use `docker ps -a`, `docker logs`, `docker inspect`, `docker exec`, and `docker stats`.

---

25. Practical Project: Django + PostgreSQL + Docker

Build a portfolio project containing:

- Django REST Framework
- PostgreSQL
- Dockerfile
- compose.yaml
- `.dockerignore`
- Environment-based configuration
- Database migrations
- Gunicorn
- API tests
- Persistent volume
- GitHub Actions CI
- Deployment documentation

Workflow

```
docker compose config

docker compose up -d --build

docker compose ps

docker compose exec web python manage.py migrate

docker compose exec web python manage.py createsuperuser

docker compose logs -f web
```

Browser:

```
http://localhost:8000
```

---

26. Learning Roadmap

Phase 1: Fundamentals

- Docker installation
- Images and containers
- Essential commands
- Port mapping

Phase 2: Dockerfile

- FROM, RUN, COPY
- CMD and ENTRYPOINT
- Build cache
- .dockerignore

Phase 3: Storage and Networking

- Named volumes
- Bind mounts
- Bridge networks
- Container DNS

Phase 4: Docker Compose

- YAML configuration
- Django + PostgreSQL
- Environment variables
- Health checks

Phase 5: Production

- Gunicorn
- Non-root user
- Multi-stage builds
- Security and backups

Phase 6: Deployment

- Image registry
- VPS or cloud deployment
- CI/CD

- Monitoring

---

27. Official Documentation

Docker Get Started:

https://docs.docker.com/get-started/

Dockerfile Reference:

https://docs.docker.com/reference/dockerfile/

Docker Compose:

https://docs.docker.com/compose/

Docker Networking:

https://docs.docker.com/engine/network/

Docker Volumes:

https://docs.docker.com/engine/storage/volumes/

Django Deployment Checklist:

https://docs.djangoproject.com/en/stable/howto/deployment/checklist/

Final Advice

প্রথমে Dockerfile, image build, container commands এবং port mapping practice করো।

তারপর Django + PostgreSQL + Docker Compose project তৈরি করো।

সবশেষে production deployment, security, CI/CD এবং monitoring শিখো।

এই roadmap অনুসরণ করলে Python/Django Backend Developer হিসেবে Docker-এর গুরুত্বপূর্ণ development ও deployment workflow শিখতে পারবে।